

research, we focus on advancing LULC classification through the integration of object detection and image segmentation techniques. An object-based classification is followed as opposed to pixel-based classification in the traditional LULC mapping methods. LULC maps for India are commonly available at 10m or greater resolution, but producing higher-resolution maps is challenging due to increased heterogeneity, necessitating an object-based approach which takes into account the shape, texture and context of the land cover features. The level of detail within each pixel also increases and a single pixel may represent mixed elements from the land cover features resulting in a possible misclassification. [1] A pixel-based classification method is also unable to take spatial relations and contextual information about the features into account thus yielding a need for an object-based approach.[2] This study addresses this challenge by using Cartosat-3 data with a 0.45m resolution, by leveraging the strengths of an object-based classification. The project is guided by two primary objectives- (1) performing object detection on the satellite images for obtaining bounding boxes around different classes of objects (2) obtaining segmentation masks of objects inside the bounding boxes using an image segmentation model. A key innovation in our approach is the application of the Segment Anything Model (SAM) by MetaAI, to satellite imagery for the purpose of image segmentation. Image segmentation involves partitioning an image into distinct regions, each corresponding to a different object or area. The zero-shot inference capabilities of SAM proved to have a game changing impact on the segmentation of images, thereby leading us to implement it on satellite imagery as well. An instance segmentation of images was obtained, i.e., the model uniquely identified each instance of an object rather than performing semantic segmentation, which involves classifying them into separate classes, required for LULC mapping. To classify these instances into their particular class, a clustering algorithm—DBSCAN—was used in order to provide labels to the segments. However, this density-based clustering did not provide satisfactory results. Finally, an object detection approach was decided, which would serve as a precursor to segmentation. Object detection is a key computer vision technique that identifies and locates objects within images or videos. It performs two main tasks: detecting the presence of objects belonging to specific classes (such as people, cars, or animals) and determining their positions by drawing bounding boxes around them. For the purpose of this research, YOLOv8 was chosen as the object detection model owing to its recognized accuracy in the field of computer vision. Once the segmented masks are obtained, they can be exported as a GeoPackage file. The results revealed that while this approach was able to fabricate accurate classification maps for discrete objects such as buildings, vacant plots, and agricultural patches, this strategy could not be used on continuous features such as rivers, roads, and forests. This paper details the methodology, implementation, and results of LULC classification using the YoloV8 and the Segment Anything Model on the Cartosat-3 images of study area which was chosen to be parts of Delhi with varied characteristics. This research was carried out as a part of a 2-month Research Internship at Regional Remote Sensing Centre- North, NRSC, ISRO, New Delhi. This paper presents a novel approach for LULC classification and further discusses the future scope and possible improvements.

2 Materials and Methods

2.1 Dataset Preparation

2.1.1 The xView Dataset

After an extensive search for datasets to train an object detection model, the xView dataset was selected. xView is a publicly available annotated dataset containing over 1 million object instances across 60 classes closely related to built-up. It covers about 1,400 km² of imagery and contains a total of 860 images of resolution 0.3 meters, collected from the WorldView-3 satellites, and features multiple object types with bounding box annotations in geoJSON format. It was divided into training, validation, and test sets in the ratio of 80:10:10 in order to train the object detection model. From Figure 1, the limitations of the model become apparent. It can be seen that the model

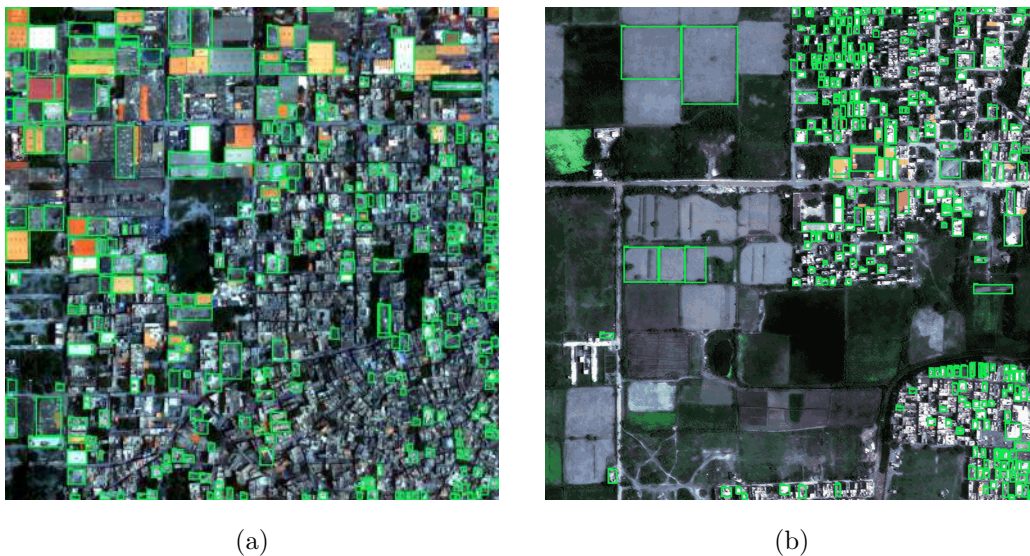


Figure 1: Results obtained by training YoloV8 on the xView dataset.

could not detect small houses in clustered areas and also misclassified Barren Land as Built-up. In addition to this, the dataset did not have enough classes to produce an LULC map. It was observed that not many datasets were a fit for our research area, Delhi, due to its clustered and unorganized built-up parts. The absence of datasets for an Indian context prompted us to develop our own.

2.1.2 Creating a Dataset

To prepare the dataset for our object detection model, Cartosat-3 images of Delhi were used. The GeoTIFF tiles were sliced into smaller images, each of size 1280 x 1280 pixels. This particular size was chosen because during the training phase, YOLOv8 resizes images according to the ‘imgsz’ hyperparameter while maintaining the aspect ratio. This hyperparameter was chosen to be 640—implying that these training images would be resized to 640 x 640 pixels—due to a constraint in computational resources. A higher value of this hyperparameter results in a higher value of the accuracy of the

model but calls for an increase in computational resources. This was concluded to be an optimal compromise and ensured that the small features in the image would not be overlooked.

A total of 124 images were labeled by drawing bounding boxes around each instance of an object using LabelMe as the annotation tool. Figure 2 includes images from the dataset labeled by us. Initially, bounding boxes were created, and labels were provided for 10 distinct classes of objects: Building, Vegetation, Metro Station, Railway Station, Agriculture & Grassland, Bare Ground/Vacant Plot, Water Body, Aircraft, Stadium, and Airport Hangar. However, some classes were subsequently removed due to the limited number of samples available or the continuous nature of certain classes (e.g., Vegetation), as object detection is more effective for discrete objects. As a result, the dataset was refined to focus on four primary classes: 1. Building 2. Agriculture and Grassland 3. Bare Ground/Vacant Plot 4. Aircraft.

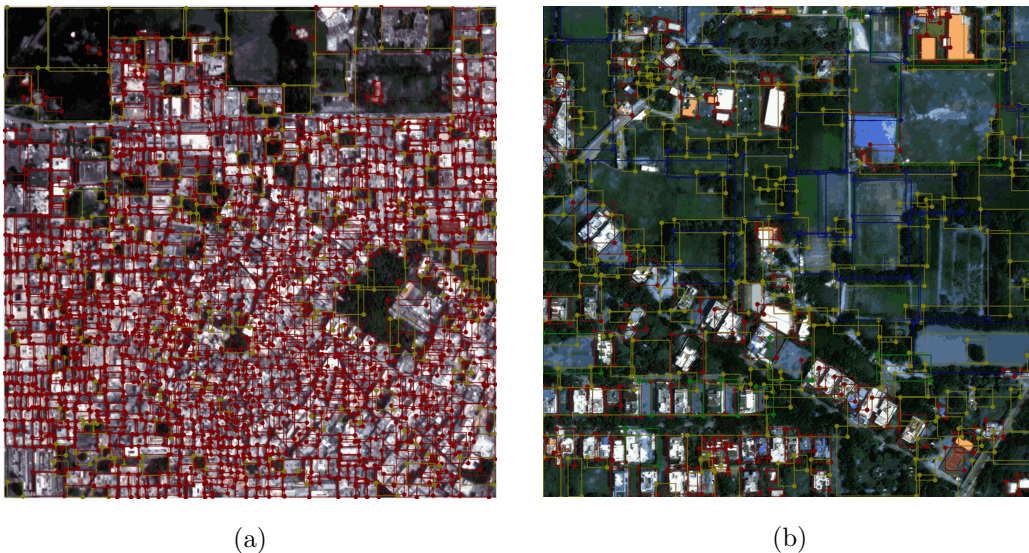


Figure 2: Snippets of the dataset created using LabelMe annotation tool.

Subsequently, data augmentation was carried out to expand the dataset size five-fold, that is, to 620 images finally. Data augmentation is a technique of artificially increasing the training set by creating modified copies of a dataset using existing data. It is performed in order to improve the accuracy of the model, to correct the imbalanced datasets, and to reduce the operational costs of labeling and modifying raw datasets. A Python library called Albumentations was employed to perform 4 different transformations on all of the images which were Rotate, Flip, Crop and RGB Shift. Figure 3 illustrates these transformations performed on our dataset.

2.2 Object detection using YOLOv8

The image segmentation model employed in this study, SAM, is capable only of instance segmentation, meaning it delineates each individual object instance within an image rather than putting a label on it. However, for land use and land cover (LULC) mapping, semantic segmentation is required, which involves classifying each pixel of the image into one of the several predefined classes.



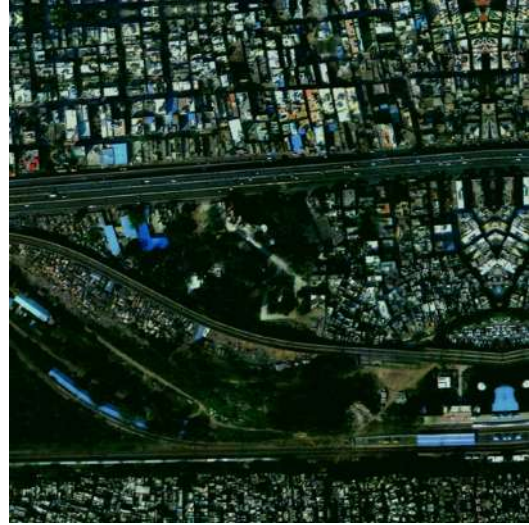
(a)



(b)



(c)



(d)

Figure 3: Data augmentation applied to the dataset: (a) Rotation, (b) Flip, (c) Crop, (d) RGB Shift.

Therefore, prior to performing instance segmentation, an object detection model is utilized to put each of the segmented instances into a category. This is done by providing the bounding box coordinates as box prompts to the segmentation model. In the realm of object detection, several models, such as YOLO (You Only Look Once), Faster R-CNN, Mask R-CNN, SSD and RetinaNet, exist and after a thorough literature review comparing different models' accuracies, YOLOV8 was selected.

2.2.1 YOLOv8 architecture

The YOLOv8 architecture is structured into 3 main components: Backbone, Neck and Head. YOLOv8 starts by dividing the input image into a grid of cells. The backbone processes the image through layers that first capture basic features like edges and textures, then progress to more complex patterns. Connecting the backbone and the head is the neck, which employs the novel C2f module, displayed in the leftmost block in the figure below. The C2f module allows the integration of high-level semantic information with low-level spatial details and uses contextual information for better accuracy, especially for smaller features. Finally, the head generates predictions, including bounding boxes, objectness scores, and class probabilities, through multiple detection modules. These predictions for each grid cell are aggregated to produce the final detection results. Further, YOLOv8 performs an anchor-free detection, which increases its robustness and efficiency. The YOLOv8 architecture is composed of certain blocks, as shown in Figure 4. The most basic block, which makes up all the other blocks at a higher level, is the convolutional block. It starts with a 2d convolutional layer, which performs convolution operations on the input image by sliding a 3x3 kernel (or a filter) over the input data to produce a feature map. This layer is followed by a 2d batch normalization layer. Batch normalization is a regularization technique to reduce overfitting by normalizing the data of the input features between 0 and unit variance to ensure they are all on the same scale and particular features do not overshadow the remaining ones. SiLU (Sigmoid Linear Unit) is the activation function used by this network. The reason for using SiLU has to do with the fact that it is not monotonically increasing like ReLU and hence introduces non-linearity while providing smooth and continuous gradients. This smoothness helps the network address the vanishing gradient problem more effectively.

The SiLU activation function is defined in the following way.

$$\text{SiLU}(x) = x \cdot \sigma(x) \quad (1)$$

where the sigmoid function $\sigma(x)$ is given by:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

2.2.2 Fine-tuning the hyperparameters

The YOLOv8 model comes in several variants: YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x. In this study, YOLOv8l (large) has been used, which features 365 layers, 43,632,924 parameters, and 43,632,908 gradients.

Due to constraints on computational resources, hyperparameters were chosen on a hit-and-trial basis for a total of 10 epochs each. The batch size (batch), which determines the number of images processed simultaneously in forward pass, was set to 8 as a compromise between speed of convergence and resources. The image size (imgsz) specifies the dimensions to which the input image is resized for output while maintaining the aspect ratio. The image size selected was 640 pixels. Pretrained was

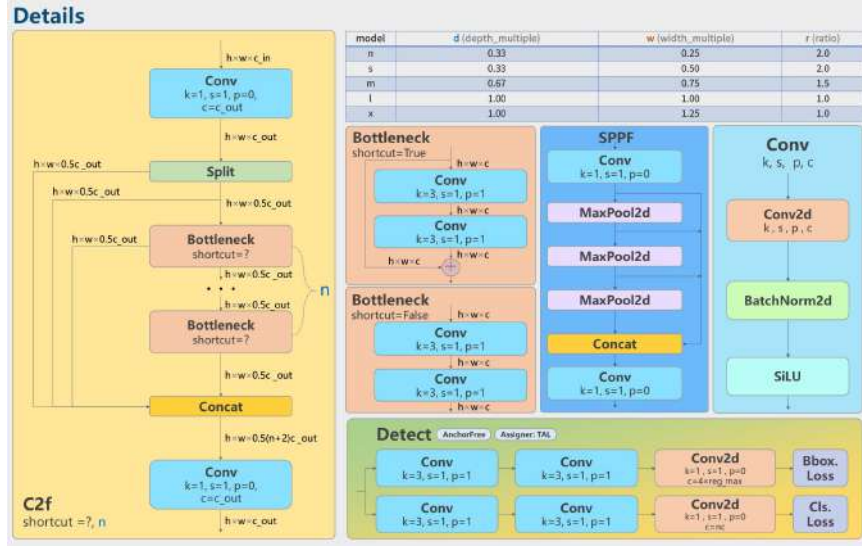


Figure 4: Details of the YOLOv8 blocks that make up its architecture.

set to false, which initialized the weights randomly rather than importing them from the pretrained model. This allowed for our model to converge to a higher mAP value as opposed to when pretrained was set to True. The learning rate (lr0) specifies how fast the model’s weights are updated. A high value can cause oscillations and failure to reach the local minima, while a low value can cause slower convergence. The learning rate in this study was set to 0.001. Momentum (Momentum=0.937) helps the model get unstuck from the local minima; adjusting this value may enhance the optimization process. The value here was set to default. The model was trained for 175 epochs, after which it started to show tendencies of overfitting in most of the classes. The mAP values for Agriculture & Grassland, Bare Land/Vacant Plots, and Planes decreased; however, the mAP for buildings increased. This suggests an overfitting for the former classes but a scope for improvement for the latter. A compromise between the two led to the selection of the weights of the model obtained after a training period of 175 epochs. The computational time for this was approximately an hour, and it was trained on the A100 GPU on Google Colab Pro. Figure 5 shows bounding boxes generated by YOLOv8 on Cartosat-3 trial images taken from Delhi. The class denoted by blue boxes represents built-up, cyan indicates Agriculture & Grassland, and white signifies Bare Ground/Vacant Plot.

2.3 Image segmentation using Segment Anything Model (SAM)

The Segment Anything Model (SAM), developed by Meta AI, is a state-of-the-art image segmentation model. The fundamental ability of SAM is to accurately detect the existence of objects in an image and segment them. When encountering uncertainty about which object to segment, SAM can generate multiple valid masks, offering a range of potential segmentations for a given object to choose from. SAM is trained on the expansive SA-1B dataset, which includes over 1 billion masks spread across 11 million carefully curated images. As a result, SAM’s zero-shot performance[3], which highlights its ability to perform highly accurate image segmentation on novel images without

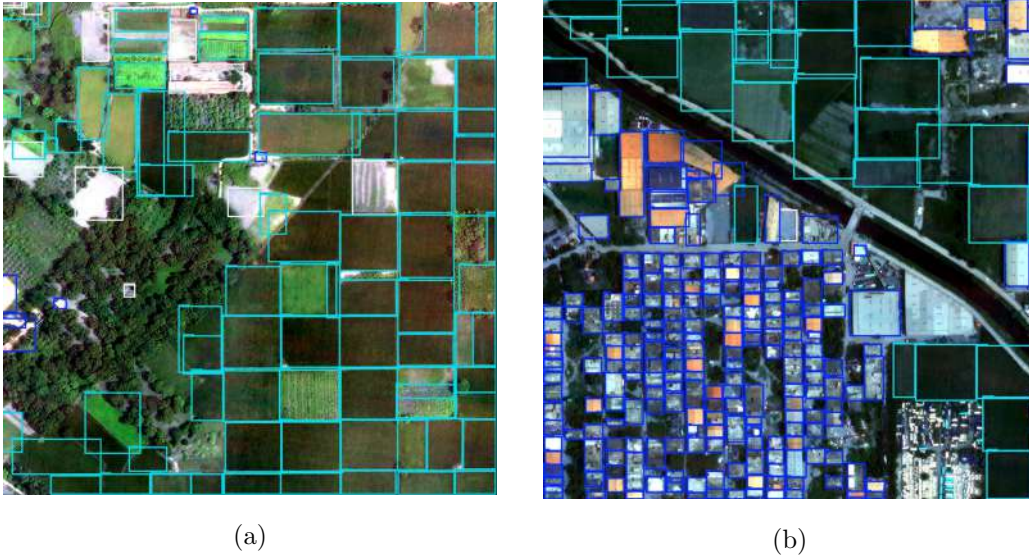


Figure 5: YOLOv8 results on Cartosat-3 trial images.

additional training, became the motivation to select this over traditional models trained that demand supervised learning. An interactive segmentation is offered by SAM, which can take a prompt as an input, allowing refinement of the segmentation process according to the user’s necessities. The prompt can be in the form of bounding boxes, points or text-based prompts, all of which provide information about the objects that are to be segmented, called upon by the Predictor method. Else, if a prompt is not given, Segment Everything mode instantiates the automatic mask generator script, segmenting all the instances in the image.

2.3.1 Architecture of SAM

The Segment Anything Model (SAM) network architecture contains three essential components: the Image Encoder, the Prompt Encoder, and the Mask Decoder, illustrated clearly in Figure 6. The core of SAM’s architecture stands on the Image encoder, which utilizes a masked autoencoder approach built on the Vision Transformer (ViT) framework, specifically the ViT-H/16 variant. This Vision Transformer processes image inputs by dividing them into 16x16 pixel patches, allowing the model to efficiently capture both local and global features within the image. The Image Encoder creates embeddings from high-resolution images in a single pass, providing a thorough representation of the image prior to the application of any prompts. Following this, is the prompt encoder—engineered to process the user inputs in the form of prompts, owing to SAM’s interactive segmentation feature. It processes two types of inputs: dense prompts and sparse prompts. For dense prompts, such as masks, the Prompt Encoder applies convolutional layers to generate embeddings, which are then merged with image embeddings through element-wise summation. This integration enhances the segmentation output by incorporating the dense prompts directly with the image data. In the case of sparse inputs—such as points, bounding boxes, and text—the Prompt Encoder utilizes positional embeddings specific to each prompt type. Text inputs are interpreted through a pre-

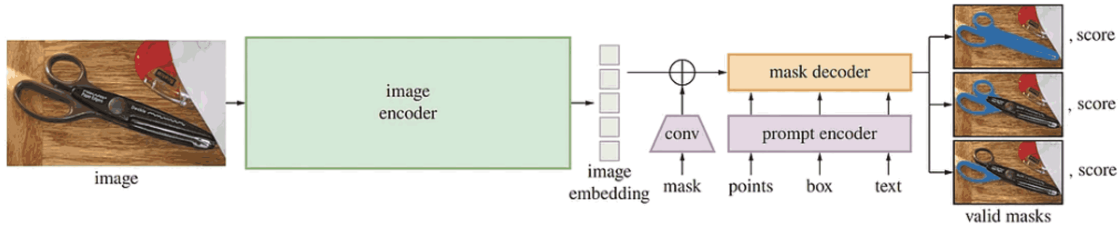


Figure 6: Architecture of Segment Anything Model

existing encoder from CLIP (Contrastive Language-Image Pre-training), translating free-form text into meaningful representations. Finally, the Mask Decoder’s essentiality arises from its capability to predict segmentation masks based on the embeddings from both the image and prompt encoders. It combines these pieces of information, along with an output token, to create the final masks.

2.3.2 Generation of Segmentation Masks for LULC mapping

To obtain segmentation masks for the Land Use/Land Cover (LULC) map, the bounding boxes around detected objects obtained from YOLOv8 were converted into a GeoJSON file, which was consequently used as a prompt input to SAM for the prediction of segmentation masks. This involves setting up a ‘SamPredictor’ object, which acts as an interface between the SAM model and the bounding boxes. The ‘SamPredictor’ is initialized with the SAM model, which is composed of its architecture and learned weights. For each bounding box, the coordinates are fed into the ‘SamPredictor’, which then generates a segmentation mask outlining the object within the box. To ensure clarity, the ‘multimask_output’ parameter is set to ‘False’, guaranteeing that only one mask is predicted per bounding box, thus eliminating repetition.

Once the masks are created, they go through a post-processing step to make them ready for visualization. This involves determining the mask’s dimensions in pixels and converting it into a 3D array to include color channels. A color array is then fed into the mask’s color channels to highlight the segmented areas on the image for improved visual clarity for LULC mapping. The segmented results are saved as a GeoPackage file in vector format.

2.3.3 Fine-tuning SAM’s Parameters

To obtain better segmentation masks in the Segment Anything Model (SAM), several parameters can be fine-tuned. These parameters help adjust the model’s sensitivity, accuracy, and reliability when predicting segmentation masks. The parameters fine-tuned in this study are detailed below. The **pred_iou_thresh** parameter sets a threshold for the predicted Intersection over Union (IoU) score, which measures how well the predicted mask overlaps with the actual object. Its value ranges from 0 to 1. Increasing this threshold can lead to higher precision by filtering out lower-quality masks. Given the densely populated areas in Delhi, a high overlap of masks was likely, so the **pred_iou_thresh** value was set to 0.5 to account for this. The **stability_score_thresh** parameter is used to ensure the reliability of the masks by measuring how stable they are under variations

in thresholding. Its value ranges from 0 to 1. Higher values here help keep only those masks that remain consistent, improving the overall robustness of predictions. Therefore, its value was set to 0.75. The **box_nms_thresh** parameter is involved in non-maximal suppression (NMS) to filter out duplicate or overlapping masks. Its value ranges from 0 to 1. Adjusting this threshold allows control over how many overlapping masks are permitted, which is crucial for handling closely packed objects. Higher values allow more overlap between masks before one is suppressed. This is useful in scenarios where objects are closely packed, so the value was set to 0.7. The **min_mask_area** parameter specifies the minimum area a mask must cover to be considered valid, which helps filter out noise by ignoring very small masks. It helps control the granularity of the segmentation masks by filtering out those that are too small to be considered significant. The value of this parameter can be any positive integer. The unit of min_mask_area is typically in square pixels. This means that the value represents the number of pixels that a mask must cover to be retained. Due to the nature of different classes covered in this model, for example, a house in a slum, the value of min_mask_area is set to 15. Table 1 summarizes the above mentioned parameters. Figure 7 displays the output from SAM by using the default parameters (b) and the fine-tuned parameters (c)

Table 1: Fine-tuned parameters for Segment Anything Model (SAM)

Parameter	Value
pred_iou_thresh	0.5
stability_score_thresh	0.75
box_nms_thresh	0.7
min_mask_area	15

3 Results

This section lays out the outcomes obtained from our proposed methodology. Initially, the metrics are listed to evaluate the accuracy in a numerical format, followed by the visual results of the model when it was applied to our study area.

3.1 YOLOv8 Performance

The performance of the YoloV8 model for object detection after being trained for 175 epochs was evaluated by employing evaluation metrics, which are highlighted below. During the prediction mode, YOLOV8 allows for the specification of a confidence threshold, which determines the minimum score required for a prediction to be considered true. Predictions with scores below this threshold are discarded. In this study, a confidence threshold of 0.25 was selected.

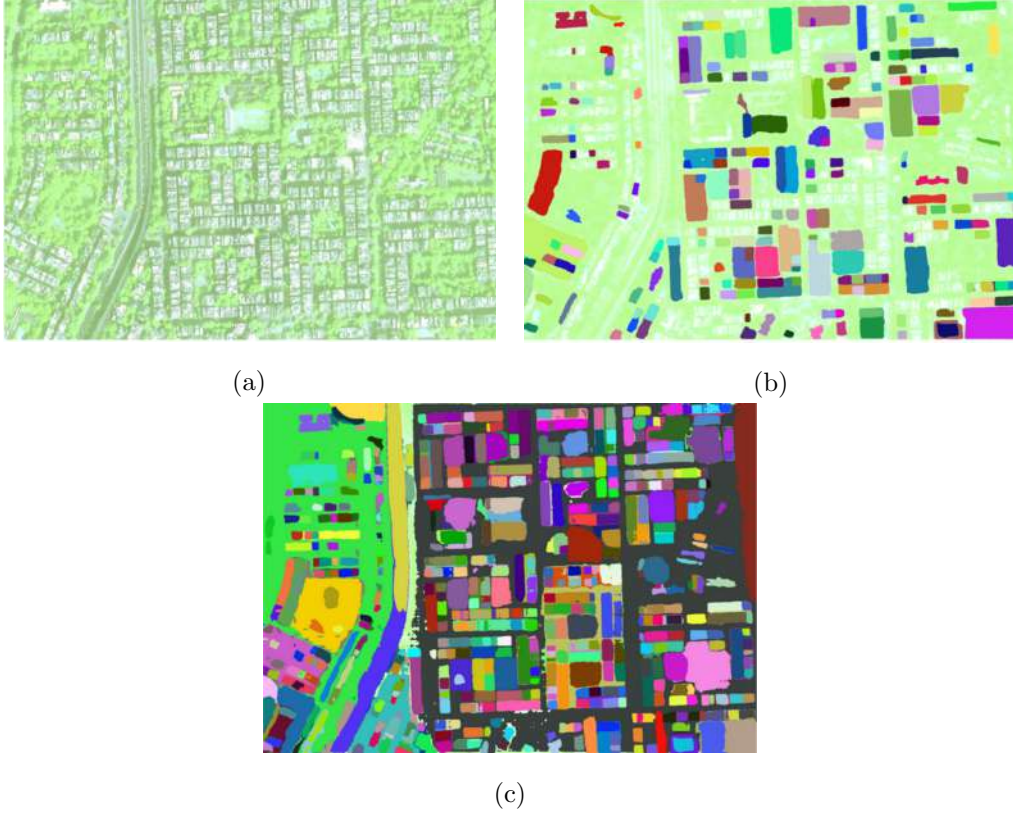


Figure 7: SAM Results. a) Original test image, b) Image segmentation result with default SAM parameters, c) Image segmentation result with fine-tuned SAM parameters as mentioned in Table 1.

3.1.1 Precision

Precision indicates the amount of positive prediction accomplished by the model. It is the ratio of true positives (TP) and the sum of true positives and false positives (FP) and is given by,

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

In Figure 8 (a), at the confidence level of 0.25, the precision is obtained to be 0.86.

3.1.2 Recall

Recall examines the data percentages that are correctly predicted. It measures the ability of the model to identify all instances of objects in the input image. It is defined as the ratio of true positives (TP) to the sum of true positives and false negatives (FN).

$$Precision = \frac{TP}{TP + FN} \quad (4)$$

In Figure 8 (b), at the confidence level of 0.25, the recall is obtained to be 0.74.

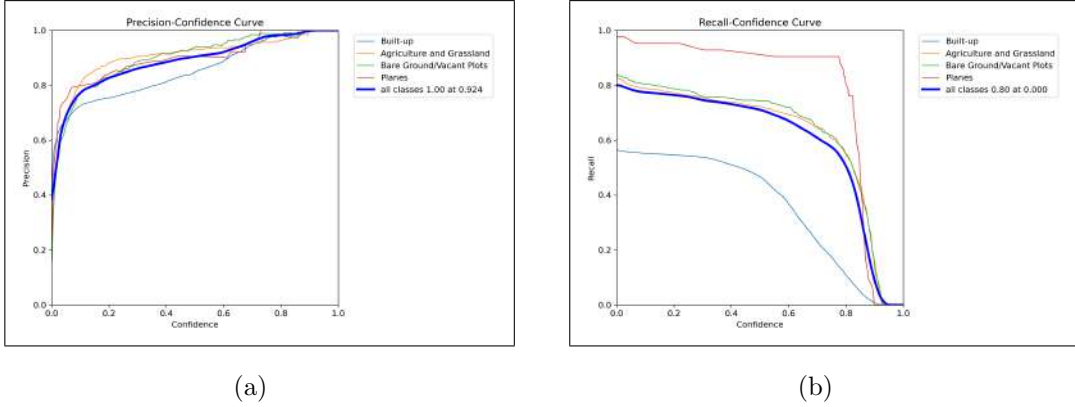


Figure 8: (a) Precision-Confidence curve, b) Recall-Confidence curve

3.1.3 Mean Average Precision (mAP)

Average precision for an object class indicates the model’s ability to precisely locate and retrieve objects accurately. Computing the area under the precision-recall curve (Figure 9 (a)) for a particular class yields the AP value for that class. Mean average precision calculates the average precision across all the classes, encapsulating the model’s performance in a single value. The mAP value is calculated over two disjoint intervals of intersection over union (IoU) threshold intervals: mAP@50 and mAP@50-95. mAP@50 measures the mean average precision at an intersection over union (IoU) threshold of 0.50, focusing on easy detections to assess model accuracy. In contrast, mAP@50-95 provides a more thorough evaluation by averaging the mean average precision across a range of IoU thresholds from 0.50 to 0.95, measuring the model’s performance for comparably challenging detections. Table 2 summarizes the mAP values for all the classes.

Table 2: Mean Average Precision values for all the classes at two distinct IoU threshold ranges.

Class	mAP@50	mAP@50-95
All	0.792	0.525
Agriculture & Grassland	0.805	0.657
Bare Ground/Vacant Plots	0.778	0.589
Built-up	0.602	0.373
Planes	0.985	0.479

3.1.4 F-1 score

F-1 score is the measurement that encapsulates both the precision and recall values. It ranges from 0 to 1, with a higher value indicating better performance. It is the harmonic mean of precision (P) and recall (R) and is given by,

$$Precision = 2 \times \frac{R \times P}{R + P} \quad (5)$$

In Figure 9 (b), at the confidence level of 0.25, the F-1 score is 0.785.

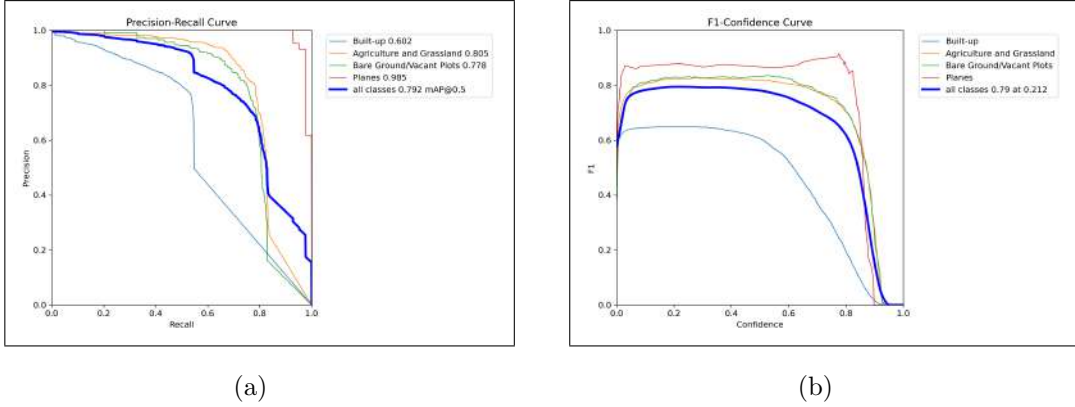


Figure 9: (a) Precision-Confidence curve, b) Recall-Confidence curve

3.1.5 Loss function

A loss function measures the difference between the model’s predictions and the actual values, providing a quantitative way to evaluate the model’s performance. During the training phase, this loss function must be minimized using optimization algorithms; this paper uses ADAM (Adaptive Moment Estimation) as its optimization algorithm. The exact reason for choosing Adam over RMS is beyond the scope of this paper and is covered in [4]. Since Adam Optimizer is frequently used in literature and is easier to implement, it was selected for this research work. In YOLOv8, the loss function is composed of two main components: regression loss, which includes CIoU loss and DFL loss, and classification loss, represented by VFL loss. [5] CIoU (Complete Intersection over Union) loss, represented as box_loss during training, measures the inconsistency between the predicted and the ground truth bounding boxes by taking into account the overlap ratio, the distance between the center points and the aspect ratio between them, thereby maximizing the localization accuracy. DFL loss (Distribution Focal Loss), or the dfl_loss during training, enhances the detection performance of the small targets. The VFL loss (Varifocal Loss), depicted as cls_loss in the training logs, measures the accuracy of object categories in the prediction box. By varying the weight of positive and negative samples, it addresses the issue of sample imbalance and sharpens the model’s emphasis on samples that are challenging to categorize and minimizes the error of classification prediction. Before the network applies backpropagation and updates its parameters, the losses with their corresponding weights are summed up, which contributes to the final loss used to train the model. Figure 10 displays all the above-mentioned losses and the number of epochs. It can be discerned that as the number of epochs increase, the losses decrease indicating convergence of the model.

$$\text{Loss} = \lambda_{\text{CIoU}} \cdot \text{CIoU} + \lambda_{\text{DFL}} \cdot \text{DFL} + \lambda_{\text{VFL}} \cdot \text{VFL} \quad (6)$$

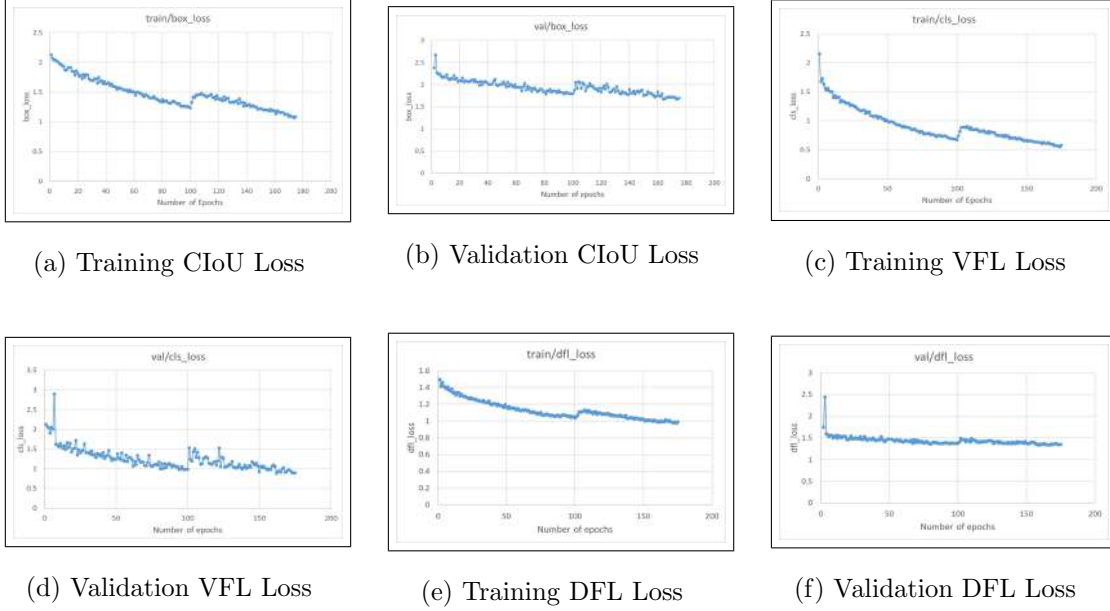


Figure 10: (a) Training CIoU Loss, (b) Validation CIoU Loss, (c) Training VFL Loss, (d) Validation VFL Loss, (e) Training DFL Loss, (f) Validation DFL Loss

3.2 Visual results

The following section presents the integrated results of YOLOv8 and the Segment Anything Model (SAM) on our study area in Figures 11, 12, and 13. The top-left image depicts the original input. The top-right image illustrates the results of YOLOv8 object detection. The bottom-left image displays the output of SAM when the bounding boxes generated by YOLOv8 were fed into it. Finally, the bottom-right image shows the combined results of both models. It can be noted that the diffuse boundaries are also taken into account which would have otherwise been neglected by using only a segmentation model.

4 Discussion

Coupling of object detection and image segmentation for LULC mapping is a technique that can only be used upon discrete objects in a satellite image. Objects of continuous nature—such as roads, rivers, and forests—fall out of scope of this technique. However, the discrete objects—such as buildings, vacant plots, agricultural patches and small water bodies—can be accurately delineated and segmented, providing an accurate measurement of their area. Object detection allows for quick identification and localization of objects, whereas image segmentation refines these detections, including diffuse boundaries as well. The precision-confidence curve indicates "all classes 1.00 at 0.924," suggesting that at a confidence threshold of 0.924, the model achieves perfect precision for all classes. This implies that the model can accurately identify instances of each class at higher confidence values. However, there might be slight variations in precision among different classes, potentially due

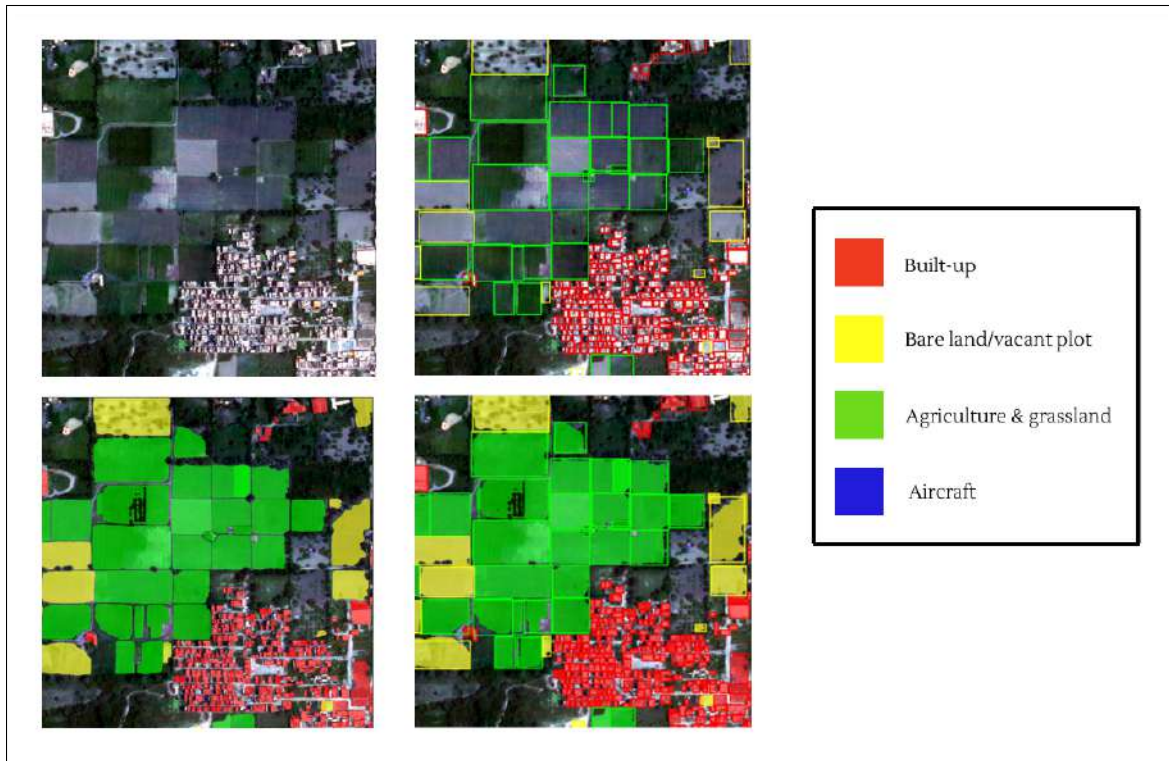


Figure 11: Result of using a box prompt in SAM derived from YOLOv8 applied to Swaroop Nagar.

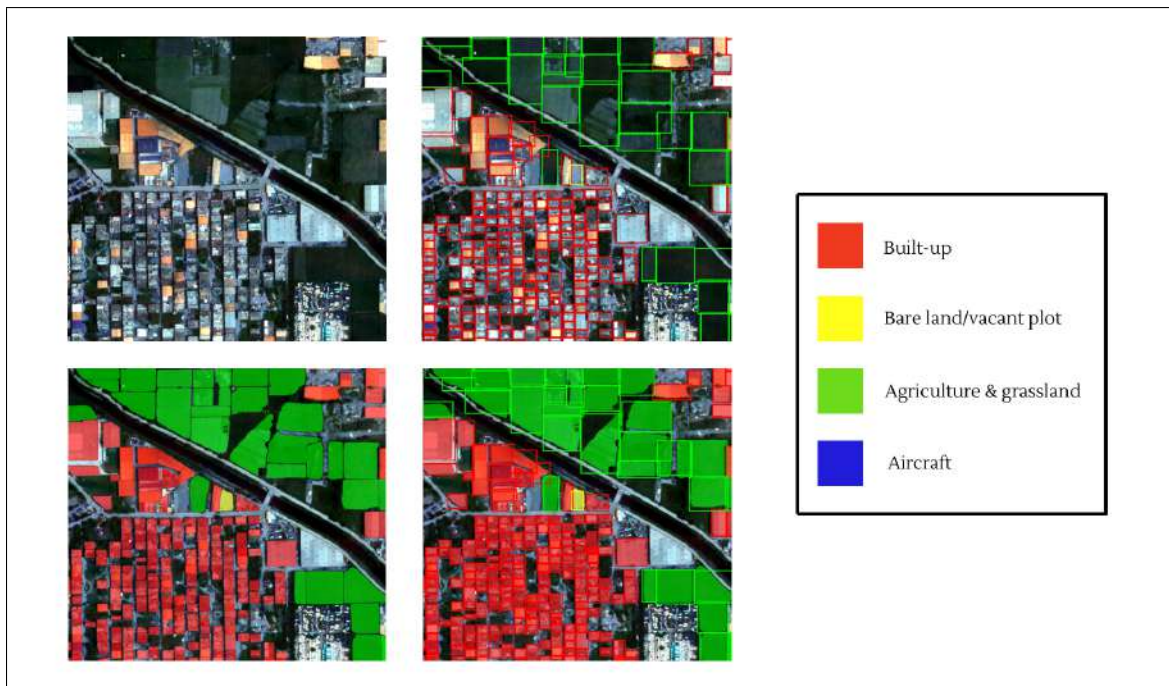


Figure 12: Result of using a box prompt in SAM derived from YOLOv8 applied to Budhpur.

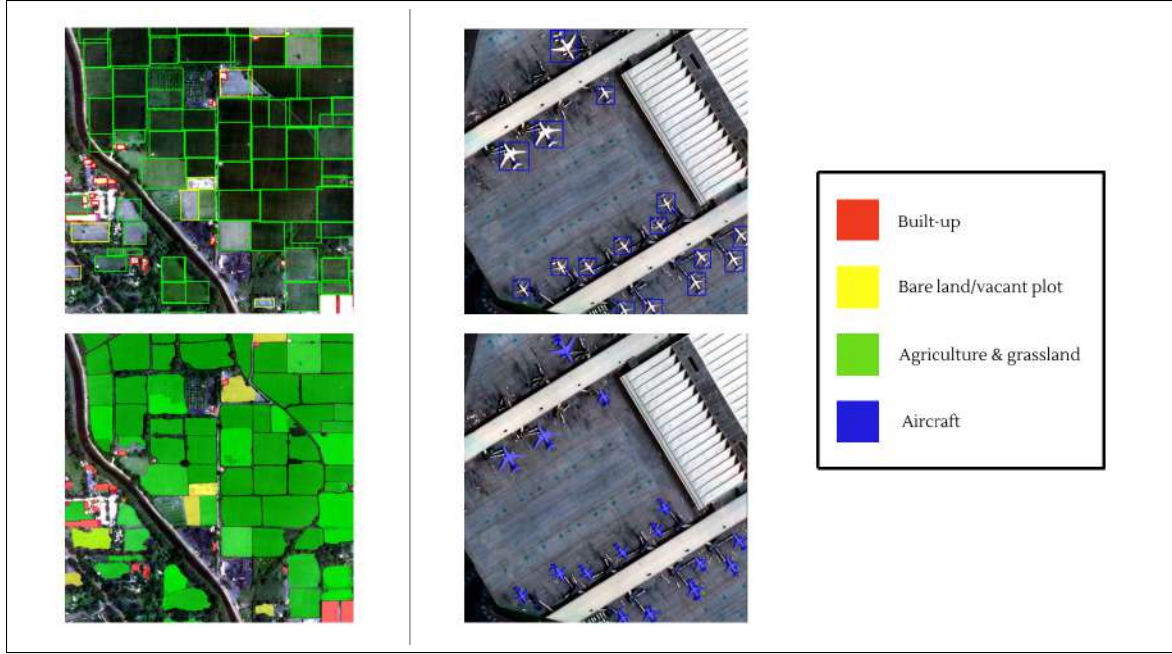


Figure 13: Result of using a box prompt in SAM derived from YOLOv8 applied to Alipur (left) and Terminal-3, IGI Airport (right). The top images show the results from YOLOv8 and the bottom images show the result from both YOLOv8 and SAM.

to factors such as class imbalance, data quality, or the inherent complexity of distinguishing certain classes. The curves for all the classes show a relatively steady increase in precision with increasing confidence, suggesting consistent performance across confidence levels until it reaches a plateau. A similar implication can be derived from the recall-confidence curve. It highlights an overall recall of 0.80 at a confidence threshold of 0.000. This suggests that the model can identify a substantial number of instances from all classes. However, a relatively steady decrease in recall with increasing confidence, suggests that the model might miss a higher proportion of instances as the confidence threshold increases. The provided loss curves visualize the training and validation loss for different components of the YOLOv8 model: `box_loss`, `cls_loss`, and `dfl_loss`. The `box_loss` curve, `cls_loss`, and `dfl_loss` curves all show a general decreasing trend for both training and validation, indicating that the model is improving in predicting the bounding boxes of objects, classifying objects correctly and predicting the objectness score, respectively. Overall, the decreasing trends in the loss curves suggest that the model is converging and learning from the training data. **Further Improvements**

for the Future

YOLOv8 was initially trained for 175 epochs, followed by an additional 75 epochs. However, during this extended training, the model began to exhibit signs of overfitting, particularly in the classes of bare land/vacant plots, agriculture & grassland, and aircraft. Despite this, the built-up class showed improved performance. Based on these observations, the plan was to continue training the model using a dataset focused solely on the built-up class, leveraging the weights from the previously trained model. Unfortunately, due to constraints in time and computational resources, this plan could not

be executed at this stage.

Acknowledgments

We would like to extend our deepest appreciation to our project supervisor, Anurag Mishra, for his continuous support, expert guidance, and valuable feedback on the project. We are also grateful to Rahul Kumar, a faculty member of BITS Pilani, for overseeing the project. We wish to acknowledge the support of BITS Pilani for providing us with this opportunity and to RRSC ISRO for supplying the resources and facilities necessary to carry out this project, with special thanks to Dr. Sameer Saran. “All authors contributed equally to the writing of the manuscript.”

References

1. Liu D and Xia F. Assessing object-based classification: advantages and limitations. *Remote Sensing Letters* 2010;1:187–94.
2. Martins VS, Kaleita AL, Gelder BK, da Silveira HL, and Abe CA. Exploring multiscale objectbased convolutional neural network (multi-OCNN) for remote sensing image classification at high spatial resolution. *ISPRS Journal of Photogrammetry and Remote Sensing* 2020;168:56–73.
3. Osco LP, Wu Q, de Lemos EL, et al. The Segment Anything Model (SAM) for remote sensing applications: From zero to one shot. *International Journal of Applied Earth Observation and Geoinformation* 2023;124:103540.
4. Reddi SJ, Kale S, and Kumar S. On the Convergence of Adam and Beyond. *ArXiv* 2018;abs/1904.09237.
5. Zhao W, Yao X, Wang B, et al. A Visual Detection Method for Train Couplers Based on YOLOv8 Model. In: *The 8th International Conference on Advances in Construction Machinery and Vehicle Engineering*. Ed. by Halgamuge SK, Zhang H, Zhao D, and Bian Y. Singapore: Springer Nature Singapore, 2024:561–73.